Step 1: Install and Import Required Libraries

First, we ensure `spacy`, `pandas`, and `scikit-learn` are installed, and download the `en_core_web_sm` model.

```
#MOHANASUNDARAM
#241801169
!pip install -q spacy pandas scikit-learn
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 29.1 MB/s  0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Step 2: Load and Prepare the Dataset

We will load a subset of the Amazon Fine Food Reviews dataset directly from a public URL, handle missing values, and limit the dataset to 1,000 records for fast processing.

```
import pandas as pd

# Load dataset from a reliable public CSV source of Amazon Reviews (or a fallback sample)
try:
    url = "https://raw.githubusercontent.com/numenta/NAB/master/data/realKnownCause/amazon_a
    # Using a known public NLP sample for reliable demonstration
    url_reviews = "https://raw.githubusercontent.com/skandavivek/web-mining/main/Amazon_Revi
    df = pd.read_csv(url_reviews)
    # Map typical review column names if needed
    if 'ReviewText' in df.columns:
        df['Text'] = df['ReviewText']
    elif 'review_body' in df.columns:
        df['Text'] = df['review_body']
except Exception as e:
    print("Could not load external URL, creating a synthetic sample dataset.")
    # Synthetic fallback dataset matching Amazon Fine Food Reviews schema
    data = {
        'Id': range(1, 11),
        'ProductId': ['B001E4KFG0']*10,
        'UserId': ['A3SGXH7AUHU8GW']*10,
        'ProfileName': ['delmartian']*10,
        'HelpfulnessNumerator': [1]*10,
        'HelpfulnessDenominator': [1]*10,
        'Score': [5, 1, 4, 2, 5, 5, 3, 5, 4, 2],
        'Time': [1303862400]*10,
        'Summary': ['Good buy', 'Bad', 'Great', 'Okay', 'Tasty', 'Yummy', 'Average', 'Love i
        'Text': [
            "I have bought several of the Vitality canned dog food products and have found t
```

```
                "Product arrived labeled as Jumbo Salted Peanuts... the peanuts were actually sm
                "This is a confection that has been around a few centuries. It is a light, pillo
                "If you are looking for the secret ingredient in Chex Mix, this is it!",
                "Great healthy snack! My kids love these organic fruit snacks. Very tasty!",
                "These cookies are absolutely delicious. They are soft, chewy, and full of flavo
                "The taste is okay, but it is way too sweet for my preference.",
                "Highly recommend this coffee. It has a rich aroma and a very smooth finish.",
                "These gluten-free crackers are amazing. They taste just like regular wheat crac
                "Terrible customer service and the product was expired. Extremely disappointed."
            ]
        }
        df = pd.DataFrame(data)

    # Clean missing texts and limit to 1000
    df = df.dropna(subset=['Text'])
    df = df.head(1000).reset_index(drop=True)

    print(f"Dataset loaded successfully with {len(df)} records.")
    display(df[['Score', 'Text']].head())
```

```
Could not load external URL, creating a synthetic sample dataset.
Dataset loaded successfully with 10 records.
```

	Score	Text
0	5	I have bought several of the Vitality canned d...
1	1	Product arrived labeled as Jumbo Salted Peanut...
2	4	This is a confection that has been around a fe...
3	2	If you are looking for the secret ingredient i...
4	5	Great healthy snack! My kids love these organi...

Step 3: Preprocess Text Using spaCy

We will define a preprocessing function using spaCy to tokenize, lowercase, filter non-alphabetic tokens, remove stopwords, and lemmatize.

```
import spacy

nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])

def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    doc = nlp(text.lower())
    clean_tokens = [
        token.lemma_
        for token in doc
        if token.is_alpha and not token.is_stop
    ]
    return " ".join(clean_tokens)

print("Preprocessing reviews... This might take a moment.")
df['Cleaned_Text'] = df['Text'].apply(preprocess_text)
display(df[['Text', 'Cleaned_Text']].head())
```

```
Preprocessing reviews... This might take a moment.
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	buy vitality can dog food product find good qu...
1	Product arrived labeled as Jumbo Salted Peanut...	product arrive label jumbo salt peanut peanut ...
2	This is a confection that has been around a fe...	confection century light pillowy citrus gelatin
3	If you are looking for the secret ingredient i...	look secret ingredient chex mix
4	Great healthy snack! My kids love these organi...	great healthy snack kid love organic fruit sna...

Step 4: Convert Reviews to TF-IDF Vectors

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['Cleaned_Text'])

print(f"TF-IDF Matrix shape: {tfidf_matrix.shape}")
```

```
TF-IDF Matrix shape: (10, 67)
```

Step 5: Information Retrieval System (Process Query, Compute Similarity, & Retrieve)

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def search_reviews(query, top_k=5):
    # 1. Preprocess Query
    cleaned_query = preprocess_text(query)
    print(f"Original Query: {query}")
    print(f"Cleaned Query: {cleaned_query}\n")

    # 2. Transform Query to TF-IDF Vector
    query_vector = vectorizer.transform([cleaned_query])

    # 3. Compute Cosine Similarity
    similarity_scores = cosine_similarity(query_vector, tfidf_matrix).flatten()

    # 4. Sort and Get Top K
    top_indices = similarity_scores.argsort()[::-1][:top_k]

    # 5. Display Results
    results = []
    for idx in top_indices:
        score = similarity_scores[idx]
        if score > 0:
            results.append({
                'Index': idx,
                'Similarity Score': score,
                'Rating (Score)': df.loc[idx, 'Score'] if 'Score' in df.columns else 'N/A',
                'Review Text': df.loc[idx, 'Text']
```

```
            })

    if not results:
        print("No matching reviews found.")
        return pd.DataFrame()

    results_df = pd.DataFrame(results)
    return results_df

# Example Search
results = search_reviews("tasty healthy dog food", top_k=3)
display(results)
```

```
Original Query: tasty healthy dog food
Cleaned Query: tasty healthy dog food
```

	Index	Similarity Score	Rating (Score)	Review Text
0	0	0.341930	5	I have bought several of the Vitality canned d...
1	4	0.301511	5	Great healthy snack! My kids love these organi...

Next steps: Generate code with `results` New interactive sheet